Quantstamp

Solv Protocol - BTC+/ SolvBTC Channels

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Bridge
Timeline	2026-09-08 through 2026-09-11
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Source Code	• solv-finance/solv #70f93c8
Auditors	• Zeeshan Meghji Senior Auditing Engineer • Jennifer Wu Auditing Engineer

Documentation quality	High
Test quality	High
Total Findings	9 Fixed: 7 Acknowledged: 2
High severity findings	0
Medium severity findings	0
Low severity findings	6 Fixed: 4 Acknowledged: 2
Undetermined severity findings	0
Informational findings	3 Fixed: 3

Summary of Findings

Quantstamp reviewed the smart contracts implementing the SolvBTC and BTC+ mainnet minting channels. The system allows authorized minters to submit verifier-signed Bitcoin deposit information and uses a shared registry to prevent the same Bitcoin transaction from being processed more than once. SolvBTC issuance is routed through a rate-limited mint gateway, while BTC+ issuance additionally transfers SolvBTC reserves to a configured vault and converts them into BTC+ shares using the current NAV. The system also includes configurable rate limits and pause controls intended to bound issuance and provide emergency containment.

The most significant findings relate to the design and configuration of the system's rate-limiting and minting controls. In particular, SOLV-1 identifies that the linear-decay rate limiter behaves as a continuously refilling token bucket and can permit almost twice the stated window limit over an interval just shorter than the configured window. Other findings address the absence of a minimum window duration, insufficient safeguards around BTC+ NAV freshness and movement, the ability to reset outstanding rate-limit usage through channel reauthorization, and the possibility for independently configured pause controllers to drift from the intended topology. The Auditor Suggestions section includes additional recommendations to improve observability, configuration clarity, authorization lifecycle handling, and code maintainability.

Several important recommendations are included in the Operational Considerations section of the report. The security of the broader system depends heavily on external components and operational processes, including the Bitcoin verifier, NAV oracle, SolvBTC and BTC+ contracts, reserve custody, replay-registry configuration, and governance of the various channel and pause-controller roles. We therefore recommend strong controls around verifier signing, continuous reconciliation of Bitcoin deposits and reserve backing, careful management and monitoring of rate limits and authorized callers, well-tested emergency pause procedures, and robust multisignature and timelock controls for sensitive administrative actions. The formal scope of this audit was limited to the contracts contained in the repository's `src` folder; the external SolvBTC and BTC+ contracts, oracle implementations, verifier infrastructure, and other off-chain dependencies were not included within the audit scope.

The codebase is well tested, with approximately `98.86` % line coverage and `95.40` % branch coverage across the contracts under review. The suite includes unit tests, integration and fork-based testing, and fuzz testing. The documentation provided for the review was comprehensive. A small number of discrepancies between the documentation and implementation were identified during the audit and are noted in the relevant findings and recommendations throughout the report.

Fix-Review Update 2026-09-23:
The development team fixed all but two issues in the initial report. The remaining two issues (SOLV-1 and SOLV-3) were acknowledged as being part of the intended design. All auditor suggestions were also implemented. Extensive updates were made to the documentation to keep it consistent with the implementation. During the fix review, the auditing team identified one new issue introduced by the fixes: SOLV-9. The new issue involves reauthorized channels remaining blocked beyond the documented time window.

Fix-Review Update 2026-09-30:
The development team has implemented a fix for SOLV-9. All issues in the report have now been accounted for through fixes or acknowledgements.

ID	DESCRIPTION	SEVERITY	STATUS
SOLV-1	Linear-Decay Rate Limiting Permits Almost Twice the Stated Window Limit	Low	Acknowledged
SOLV-2	Missing Minimum Window Duration Permits an Arbitrarily High Issuance Rate	Low	Fixed
SOLV-3	Btc+ Minting Lacks Oracle Freshness and NAV Movement Safeguards	Low	Acknowledged
SOLV-4	Revoking and Re-Authorising a Channel Resets Its Outstanding Usage	Low	Fixed
SOLV-5	Independently Mutable `PauseController` References Can Drift From the Intended Topology	Low	Fixed
SOLV-6	Events Report the Quoted NAV Instead of the Execution NAV	Informational	Fixed
SOLV-7	Registry Documentation Understates the Owner's Indirect Write Authority	Informational	Fixed
SOLV-8	Mint Signatures Are Transferable Between Authorized Minters	Informational	Fixed
SOLV-9	Reauthorized Channels Can Remain Blocked Beyond the Documented Recovery Window	Low	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.

2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

The scope is restricted only to the contracts under the `src` folder. The SolvBTC, BTCPlus, and oracles are not part of the audit scope. Verifiers and off-chain components are not included within the audit scope.

Files Included

Repo: `https://github.com/solv-finance/Solv4.0`

Included Paths: `src/`

Extensions: `sol`

Operational Considerations

External Dependencies

The SolvBTC reserve backing BTC+ remains under external custody. The depositor verifies that each deposit reaches the configured vault before issuing BTC+, but it cannot restrict subsequent movements from that vault or continuously enforce reserve sufficiency. Operators should reconcile SolvBTC received and held by the vault against BTC+ issuance and investigate unexpected transfers or backing discrepancies promptly.

External token upgrades can invalidate assumptions made by the immutable contracts. SolvBTC and BTC+ are upgradeable dependencies whose behavior may change independently of this system. Minting should be paused before either token is upgraded, and integration tests should be rerun against the resulting implementation before operations resume, including checks of minting, transfers, decimals, NAV queries, and share conversion.

Verification

Minting security depends heavily on the verifier. The verifier is responsible for checking transaction existence, transaction-hash canonicalization, deposit amount, confirmation depth, and inclusion of the supplied `OP_RETURN` before signing. The verifier must bind the correct transaction hash, recipient, amount, token, chain, and expiry to each authorization. Operators should independently reconcile signed requests against Bitcoin deposits, maintain confirmation and reorganization policies, prevent duplicate signing across channels and chains, and pause minting upon anomalous or inconsistent verifier output.

Verifier address behavior must remain compatible with OpenZeppelin's signature validation. If an EOA verifier acquires code through EIP-7702, `SignatureChecker` begins treating it as an ERC-1271 signer and does not fall back to ECDSA. Unless the delegated implementation supports ERC-1271 correctly, all signatures fail closed. The verifier's code status should therefore be monitored, Fordefi should confirm that its verifier address will not sign an EIP-7702 authorization, and any contract-based verifier must implement ERC-1271 from the outset.

Minting

Wrong-channel consumption cannot be corrected on chain. Both channels share `BTCTxRegistry`, and a consumed `btcTxHash_` cannot be released. If a transaction is processed through the wrong channel, it cannot later be minted through the intended channel. Operations should reconcile consumed hashes against their Bitcoin transactions because correction requires an out-of-contract process.

OP_RETURN determines the intended minting channel. Each channel checks that the supplied `opReturn_` encodes its destination chain, token, and recipient. The verifier must confirm that this payload is the OP_RETURN included in the Bitcoin transaction. Otherwise, the same `btcTxHash_` can be signed with a different payload and consumed through BTCPlus instead of SolvBTC, or vice versa.

Large Bitcoin deposits may exceed the minting limits. A mint must fit within the submitting minter's `minterCaps` and the gateway's `maxSingleAmount`. BTCPlus mints must also fit within the depositor's `maxSingleAmount`. Waiting restores window allowance but does not bypass a single-transaction limit. The contracts do not support partially minting one `btcTxHash_`, and replay protection prevents the same hash from being used for multiple successful mints. Since `maxSingleAmount` must be lower than `MAX_WINDOW_AMOUNT`, which is fixed at 100 BTC, a deposit of 100 BTC or more cannot be processed through these paths. Operations should enforce the effective maximum before users send Bitcoin and maintain a recovery process for oversized deposits.

Emergency Response

The pause mechanism requires rapid operational response. `pauseMint()` is the system's only immediate on-chain containment mechanism, particularly if the gateway or another component fails while slower administrative actions remain pending. The pause function should be continuously available, monitored, and exercised through regular incident-response rehearsals with clear criteria for pausing and resuming.

Operators must distinguish targeted and system-wide pauses. The automated tier has a separate controller so it can be stopped without interrupting the manual tier, while pausing the controller used by the gateway stops every minting route. Incident procedures should identify the correct controller for each scenario and confirm that all contracts reference the intended controller.

The burn pause functions currently stop nothing. No in-scope contract reads `burnPaused()`, so calling `pauseBurn()` emits an apparently successful pause event without affecting current operations. The function should not be used as an incident response control. Because its state persists, any future redemption integration that reuses a controller must inspect and normalize the existing burn-pause state before going live.

A verifier incident requires rotation in addition to pausing. Pausing prevents immediate execution but does not invalidate previously issued signatures. Furthermore, the 14-day validity restriction bounds remaining validity at execution rather than the age of a signature, so dormant

signatures may become executable much later.

Retired channels should be deauthorized at every integration point. Disabling a current channel at the gateway—or, for a BTC+ channel, at the depositor—causes its mint transaction to revert and rolls back any registry consumption performed earlier in that transaction. Nevertheless, its registry authorization should also be revoked as part of complete retirement, preventing accidental reactivation and ensuring that no obsolete or unexpectedly behaving authorized contract retains an independent permission to consume transaction hashes.

Configuration and Lifecycle

Configured registry callers can consume arbitrary replay keys. `BTCTxRegistry.consume()` can be called by any address authorized through `setChannel()`. The registry checks that `btcTxHash_` is nonzero and unused, but it does not check that the caller is a real channel or that the hash corresponds to a valid Bitcoin transaction. An incorrectly authorized address can mark arbitrary replay keys as consumed and make the corresponding deposits unmintable. Operations should treat registry authorization as high impact, authorize only intended channel contracts, and monitor `Consumed` events for unexpected callers or transaction hashes.

SolvBTC minting relies on gateway caller configuration. `SolvBTCMintGateway` holds the SolvBTC minter role, but `mint()` receives only a recipient and amount. It does not receive a `btcTxHash_` or verify a verifier signature. Assigning an address a nonzero window through `setChannelRateLimit()` authorizes it to mint SolvBTC within its limits. The intended channel contracts verify the Bitcoin deposit before calling the gateway, but the gateway does not enforce that its caller is one of those contracts. Operations should treat `setChannelRateLimit()` as a grant of SolvBTC minting authority, authorize only the intended channel contracts, and reconcile gateway `Minted` events with Bitcoin transaction hashes consumed by the channels. Each configured caller has an independent allowance that replenishes over time.

Rate-limit exposure must be calculated across every authorized channel. Limits are maintained per channel rather than globally, and the current implementation behaves as a continuously refilling token bucket. Operators should calculate aggregate exposure using the actual refill semantics, account for every enabled channel, and reassess the total whenever a channel or tier is added.

Rate-limit reductions preserve outstanding usage and may delay recovery. Both in-place updates and revoke-then-reauthorize preserve outstanding usage. Usage does not decay while a channel is revoked and, once new parameters are installed, it decays at the newly configured rate. Reducing `maxWindowAmount` below outstanding usage can leave the channel without minting capacity for one window. Before executing a reduction, operators should inspect the channel's current `amountUsed`, calculate the expected recovery time, account for pending Bitcoin deposits, and coordinate changes to both the gateway and depositor for BTC+ channels.

Replacement deployments must preserve the shared replay history. Because the channels are non-upgradeable, fixes require replacement deployments. Replacement channels must continue using the existing `BTCTxRegistry`, and retired channels must be disabled at the registry as well as the gateway and depositor. Otherwise, a retired channel may remain able to consume deposit hashes even after it can no longer mint.

An empty registry depends on the legacy minter having processed no deposits. The deployment design relies on the stated premise that the previous `SolvBTCMainnetMinter` never entered production and has no historical deposit records. This should be confirmed against available on-chain events and operational records before launch because the new registry has no mechanism for importing previously consumed transaction hashes.

Ownership must not be renounced while configuration remains necessary. The contracts inherit `renounceOwnership()`, which would permanently disable verifier replacement, limit updates, vault changes, and channel management. Operational tooling and transaction-review procedures should reject unexpected ownership-renunciation calls and alert if one is scheduled or submitted.

Key Actors And Their Capabilities

Verifier. Each channel has a single verifier whose signature authorizes the Bitcoin transaction hash, recipient, amount, token, NAV where applicable, and expiry. A verifier cannot execute a mint without an authorized minter, but incorrect or malicious signatures can enable unsupported issuance when submitted by one. The channel owner can replace the verifier; this takes effect immediately at the contract level, subject to any delay imposed by the owner. Verifiers should use hardened signing infrastructure, transaction-level policy controls, independent Bitcoin validation, and auditable signing records. Contract verifiers must correctly implement ERC-1271, while EOA verifiers must avoid incompatible EIP-7702 delegations.

Minters. An address with a non-zero `minterCaps` entry may submit valid verifier-signed requests. The cap is a per-transaction maximum rather than a cumulative allowance. A compromised minter cannot forge or alter a request, but it can control submission timing, front-run a legitimate request, and consume the associated allowance and transaction hash. Channel owners add, update, or remove minters by changing their caps; rotating a minter requires explicitly setting the old minter's cap to zero. High-capacity minters should be controlled by a multisig or comparably strong approval system, while automated minters should receive only the minimum capacity required for routine operation.

Channel owners. The owner of each channel can replace its verifier and configure minter caps. The BTC+ channel owner can additionally recover the full balance of any ERC-20 token held by that channel. The pause controller, gateway, depositor, registry, and token addresses are immutable and cannot be changed by the channel owner. A malicious or misconfigured channel owner can authorize a bad verifier, grant mint-submission rights, or, for the BTC+ channel, sweep residual token balances. If verifier and minter controls are placed under unsafe parties, invalid Bitcoin deposits can be signed and submitted, although issuance remains subject to the downstream gateway and depositor limits.

`SolvBTCMintGateway` **owner.** The gateway owner determines which callers may mint SolvBTC and configures their transaction and rate limits. The pause controller is fixed at construction. Authorizing an unintended address grants it direct, rate-limited access to SolvBTC minting, while raising limits increases issuance exposure. Because outstanding usage survives updates and revocation, reducing an allowance below current usage may suspend that caller for one window. `MAX_WINDOW_AMOUNT` limits each caller separately, so total minting capacity grows with the number of authorized callers.

`BTCPlusDepositor` **owner.** The depositor owner configures authorized channels and their limits, and selects the vault that receives SolvBTC reserves. Changing the vault redirects all subsequent reserve deposits, so an incorrect or hostile vault can receive the assets backing newly issued BTC+. The owner cannot directly withdraw existing reserves from the vault through the depositor.

`BTCTxRegistry` **owner.** The registry owner authorizes and revokes addresses permitted to consume Bitcoin transaction hashes. Since the registry does not validate an authorized address's code or identity, the owner can grant an arbitrary address the ability to mark transaction hashes as permanently consumed, potentially preventing the corresponding deposits from being minted. Consumed records cannot be deleted or reassigned.

Management of the protocol contract owners. Each channel, gateway, depositor, and registry uses `Ownable2Step`. The owner is assigned during construction and may be reassigned through the two-step `transferOwnership()` and `acceptOwnership()` process. Administrative actions have no inherent delay and are immediate unless the owner is itself a timelock. The documented deployment plan places ownership with a governance multisig initially and later contemplates migration to a timelock. These roles should be held by a multisig, with high-impact changes subject to delayed execution, independent transaction review, and monitoring of ownership and configuration events.

`PauseController` **owner.** The `PauseController` owner can immediately pause or resume minting for every contract connected to that controller. The automated tier has a dedicated controller, while the controller used by the gateway acts as the system-wide minting stop. The owner can also change the burn-pause flag, although no current in-scope contract reads it. PauseController ownership uses the same two-step transfer mechanism, but pausing itself has no built-in delay. This role should be held by a highly available emergency multisig capable of responding promptly and should not be placed behind a governance delay that defeats its emergency purpose. A malicious owner can halt every scoped mint and deposit path or refuse to pause them during an incident.

SolvBTC contract and its administrators. SolvBTC is an external token contract that mints when called by an address holding `SOLVBTC_MINTER_ROLE`; the gateway is intended to hold that role. The token administrator can grant or revoke minting authority, while the external factory's administrator can change the `BeaconProxy` implementation. According to the supplied documentation, SolvBTC upgrades currently pass through a 72-hour timelock. A defective or malicious implementation could change minting, balance, transfer, or access-control behavior and invalidate assumptions made by the in-scope contracts. Token administration and upgrades should be protected by multisig governance, delayed execution, public review, and coordinated integration testing.

BTC+ contract and NAV oracle. The BTC+ contract issues shares when called by an address holding its minting role; the depositor is intended to hold that role. Its share conversion functions rely on an external NAV oracle, so incorrect oracle output changes how many BTC+ shares are issued for a given SolvBTC deposit. The BTC+ administrator can grant or revoke minting authority, and the documented `BeaconProxy` upgrade authority is a Safe without an enforced delay. The oracle's precise administration and update process are outside the audit scope. BTC+ and oracle administration should use robust multi-party controls, transparent change procedures, continuous NAV monitoring, and coordination with this system before token, conversion, or oracle behavior changes.

SolvBTC reserve vault. The configured vault receives and custodies the SolvBTC backing BTC+ minted through the depositor. The depositor verifies that each individual transfer reaches the vault, but it cannot prevent the vault from subsequently moving those assets or enforce continuing reserve sufficiency. The depositor owner may replace the vault for future deposits, while control over assets already received belongs to the vault itself. The vault should be governed by a multisig with restrictive transaction policies, separation from routine mint submission, and regular reconciliation of its SolvBTC holdings against BTC+ issuance and other reserve obligations.

Authorized Gateway Callers. Addresses with a configured `SolvBTCMintGateway` rate limit can mint SolvBTC to any recipient within their per-transaction and per-window limits. The gateway does not require a verifier signature or `btcTxHash_`. An authorized caller can mint SolvBTC without corresponding Bitcoin backing, and each authorized caller receives an independent limit.

Authorized Depositor Callers. Addresses with a configured `BTCPlusDepositor` rate limit can transfer SolvBTC into the configured vault and mint the corresponding BTCPlus shares to any recipient. Each BTCPlus mint requires the caller to transfer the stated SolvBTC amount to the vault. An authorized caller can bypass channel signature, OP_RETURN, and Bitcoin transaction checks, but remains limited by its supplied SolvBTC and configured rate limit.

Authorized Registry Callers. Addresses authorized by `BTCTxRegistry` can consume any unused nonzero `btcTxHash_` and record its receiver. The registry does not validate Bitcoin transaction data. An authorized caller can consume arbitrary replay keys and prevent the corresponding deposits from being minted through another channel.

Findings

SOLV-1

Linear-Decay Rate Limiting Permits Almost Twice the Stated Window Limit

Low | Acknowledged

Update

The development acknowledged the issue and provided the following explanation:

> Acknowledged. The token-bucket behaviour is intended; the documentation that described it as a hard rolling-window cap was wrong and has been corrected. maxWindowAmount is now documented as a burst capacity with a refill rate of maxWindowAmount / window, every derived figure is recomputed as maxWindowAmount × (1 + Δt / window), and test/RateLimitBurst.t.sol reproduces your worked example. We are not adopting a sliding window: burst plus refill is the behaviour we want operationally.

The documentation was updated in commits `c5466b9`, `eac2209`, `6dcc58a`, and `0bb7b0e`. The auditing team has downgraded the severity to low.

File(s) affected: `src/utils/ChannelRateLimiter.sol`, `docs/MainnetMint-Architecture.md`, `docs/SECURITY-SELF-REVIEW.md`, `docs/IMPLEMENTATION-SPEC.md`

Description: `ChannelRateLimiter` deliberately regenerates allowance continuously by reducing outstanding usage at a rate of `maxWindowAmount / window`. This behavior is consistent with the contract's continuous-recovery comment, but it implements a token bucket rather than the hard rolling-window limit described elsewhere as a maximum amount "within one window" or "per 24h."

Starting with a full bucket, the maximum amount that can be consumed over an interval of `Δ` seconds is approximately:

```
maxWindowAmount + (maxWindowAmount × Δ / window)
```

Consequently, a channel can consume almost twice `maxWindowAmount` during an interval just shorter than `window`. This materially increases the documented loss bound if a channel's verifier and minter are compromised. Applying the same limits at both the gateway and depositor does not reduce this exposure because both buckets regenerate concurrently.

For example, consider a channel configured with:

```
maxWindowAmount = 100 BTC
maxSingleAmount = 99 BTC
window          = 24 hours
```

1. At `00:00:00`, the channel consumes `100` BTC using transactions of `99` BTC and `1` BTC, exhausting its allowance.
2. During the following `86,399` seconds, the allowance continuously regenerates.
3. At `23:59:59`, the channel has recovered:

```
100 BTC × 86,399 / 86,400 ≈ 99.99884 BTC
```

The channel consumes this recovered allowance using additional transactions, bringing total issuance during an interval shorter than `24` hours to approximately `199.99884 BTC`.

Recommendation: Clarify whether token-bucket behavior or a strict rolling-window cap is required. If continuous regeneration is intended, document `maxWindowAmount` as burst capacity, state the refill rate explicitly, update worst-case loss calculations to include the initial burst plus regenerated allowance, and lower configuration values where necessary. If issuance must never exceed `maxWindowAmount` during any rolling `window` interval, replace the linear-decay mechanism with sliding-window accounting and add an invariant test covering adversarial transaction timing.

SOLV-2
Missing Minimum Window Duration Permits an Arbitrarily High Issuance Rate

Low · Fixed

> **Update**
> The development team implemented a one-day minimum window in `70ee341e93172c384bf4fb617712d2a96294b7e2`.

File(s) affected: `src/utils/ChannelRateLimiter.sol`, `src/SolvBTCMintGateway.sol`, `src/BTCPlusDepositor.sol`

Description: `ChannelRateLimiter._setChannelRateLimit()` limits `maxWindowAmount` to `MAX_WINDOW_AMOUNT`, but accepts any non-zero `window`. Because allowance regenerates continuously at a rate of `maxWindowAmount / window`, the constant limits only the bucket's maximum capacity—not the rate at which new allowance becomes available.

For example, governance can configure a channel with a `100` BTC window amount and a one-second window. The channel can consume `100` BTC across multiple permitted transactions, recover the entire allowance after one second, and repeat this in each subsequent block whose timestamp advances by at least one second. This produces an effective refill rate of `100` BTC per second despite the immutable `100` BTC limit.

Although configuring the limit requires the owner and is expected to pass through governance controls, operator misconfiguration is within the stated threat model. An unexpectedly short window could therefore render the rate limiter ineffective at containing a compromised channel before the emergency pause is activated. Both the gateway and depositor inherit the same behavior from the shared limiter.

Recommendation: Enforce a minimum non-zero window consistent with the intended response period, such as `window >= 1 days` if the protocol requires 24-hour limiting. If shorter windows must remain configurable, impose an immutable maximum refill rate derived from both `maxWindowAmount` and `window`, and validate the resulting rate in deployment tooling and tests.

SOLV-3
Btc+ Minting Lacks Oracle Freshness and NAV Movement Safeguards

Low · Acknowledged

> **Update**
> The development team acknowledged the issue and provided the following explanation:

> Acknowledged. Oracle correctness remains an external trust assumption, and the safeguards are implemented off chain: the verifier reads navTime_ from getSubscribeNav(), refuses to sign when the NAV is older than a configured maximum age or has moved abnormally against its publication history, and operations pause BTC+ minting when publication is overdue. That service is in another repository and has no commit here. We first implemented an on-chain movement limit (46c23cc) and withdrew it (3ad17ef): sampled only at deposits, it blocked legitimate deposits after the oracle's normal daily step and still passed a stale, unchanged NAV. An on-chain freshness check is not possible because IBTCPlus exposes no update timestamp. Kept from 46c23cc: the deployment preflight now asserts that the oracle's decimals equal SolvBTC's.

Related documentation was addressed in `3ad17ef`, `46c23cc`, `0bb7b0e`, `d94e3e9`, and `1b9bb78`.

File(s) affected: `src/BTCPlusDepositor.sol`, `src/BTCPlusMainnetChannel.sol`, `src/interfaces/IBTCPlus.sol`

Description: `BTCPlusDepositor` reads the current NAV through `BTCPlus.getValueByShares()` and verifies that it is within `1` % of the NAV signed by the verifier. However, the verifier is expected to obtain its quote from the same on-chain oracle, so this comparison primarily detects movement between signing and execution; it does not independently establish that the oracle value is fresh or reasonable.

The minting path currently has no on-chain check for:

1. **Oracle freshness.** The interface `IBTCPlus` does not expose an update timestamp, so an unchanged stale NAV may continue to be accepted if the verifier reads and signs the same value.
2. **Abnormal NAV movement.** The depositor does not compare the current NAV with a previously accepted value. A sudden erroneous NAV can therefore be accepted once the verifier observes and signs it.

In the referenced oracle path, `SolvBTCYieldTokenOracleForSFT.getNav()` calls `getSubscribeNav(poolId, block.timestamp)` and discards the returned `navTime`. If no NAV is stored for the requested day, the underlying NAV oracle returns the latest stored value without limiting its age.

A falsely low NAV can cause substantially more BTCPlus shares to be minted for the SolvBTC deposited, diluting existing holders. A falsely high NAV can under-mint the recipient. The rate limiter bounds the SolvBTC input but does not directly bound the number of BTCPlus shares produced from an incorrect NAV.

Oracle correctness is an explicit external trust assumption, and exploitation requires failure or compromise of trusted infrastructure, which limits the severity. Nevertheless, freshness and movement checks would provide useful defense in depth against stale data, configuration errors, and abrupt oracle failures.

Recommendation:

1. If a reliable update timestamp is available on-chain, reject NAV observations that are zero, from the future, or older than a defined maximum age.
2. Require the off-chain verifier to retrieve the NAV timestamp and reject signing when the NAV is older than a configured maximum age. Since `SolvBTCYieldTokenOracleForSFT.getNav()` does not expose the timestamp, the verifier should obtain `navTime_` from the underlying `getSubscribeNav()` call or the corresponding NAV feed. Operations should pause BTCPlus minting when NAV publication exceeds the permitted age.
3. Additionally, implement a symmetric, time-based circuit breaker that limits how quickly the NAV may increase or decrease relative to a previously accepted value. The design should include clear initialization and controlled reset procedures for legitimate exceptional NAV corrections.

SOLV-4

Revoking and Re-Authorising a Channel Resets Its Outstanding Usage

Low Fixed

> **Update**
> The development team checkpointed the last used amount and the timestamp. Any new limits are checked against these amounts. Addressed in `f980041cb9e3969028b220bc344e6dbed0414502`.

File(s) affected: `src/utils/ChannelRateLimiter.sol`, `test/SolvBTCMintGateway.t.sol`

Description: Revoking a channel requires setting its rate-limit configuration to `(0, 0, 0)`. During this update, `_setChannelRateLimit` clamps `amountUsed` to the new `maxWindowAmount` of zero. If the same channel is subsequently re-authorised, it begins with no outstanding usage and immediately receives a fresh allowance.

For example:

1. A channel configured for `100` BTC per window consumes its entire allowance.
2. Governance revokes it using `setChannelRateLimit(channel, 0, 0, 0)`, clearing the recorded usage.
3. Governance re-authorises it with the original parameters.
4. The channel can immediately consume another 100 BTC without waiting for its previous usage to decay.

The revoke and re-authorise calls can be included in the same timelock batch. Repeating the process can refresh the allowance multiple times without waiting for the configured window, meaning cumulative issuance is not structurally bounded by `MAX_WINDOW_AMOUNT`.

This conflicts with the comment in `ChannelRateLimiter` stating that a multisig cannot raise the `100` BTC ceiling and that the limit is "structural." The configured `maxWindowAmount` never exceeds `100` BTC, but governance can repeatedly refresh that amount, so the constant does not provide a hard cumulative issuance ceiling.

Recommendation: If the limit must remain effective across revocation, preserve outstanding usage until it naturally decays or require the previous window to elapse before the same channel can be re-authorised. If resetting allowance is intentional, document re-authorisation as an operation that grants a fresh issuance allowance.

SOLV-5
Independently Mutable `PauseController` References Can Drift From the Intended Topology

Low · Fixed

> **Update**
> The development team made the `PauseController` reference in each contract immutable. As long as the address is set correctly on deployment, a configuration mismatch is no longer possible. Addressed in `8cfcdfbbdb7e101980398cd3dcb9bb58774090b3`.

File(s) affected: `src/PauseController.sol`, `src/SolvBTCMintGateway.sol`, `src/BTCPlusDepositor.sol`, `src/BTCPlusMainnetChannel.sol`, `src/SolvBTCMainnetChannel.sol`

Description: The intended pause topology uses a main controller for the gateway, depositor, and manual channels, while automated channels use a separate fast-tier controller. Because every channel ultimately mints through the gateway, the main controller is intended to remain the global kill switch.

Each consumer nevertheless stores its pause-controller reference independently and exposes an owner-only `setPauseController()` function. These setters only reject the zero address and the current address; they do not preserve the intended relationship between components. Deployment verification confirms the topology initially, but an incomplete or erroneous controller rotation can subsequently cause the live configuration to drift.

For example:

1. Governance unintentionally changes the gateway to use another unpaused controller but leaves the remaining contracts unchanged.
2. Emergency operators later pause the original main controller, expecting all minting to stop.
3. A fast SolvBTC channel checks its unpaused fast-tier controller before calling the gateway.
4. The gateway checks its newly configured controller rather than the paused main controller, allowing the mint to continue.

Similarly, changing only a fast channel's controller would prevent the fast-tier controller from selectively stopping that channel. These failures require a governance configuration error rather than permissionless exploitation, but they can silently weaken the emergency controls during an incident.

The `BTCPlusDepositor` comment stating that the depositor, gateway, and channels all read the same flag should also be updated because fast-tier channels intentionally read a different controller.

Recommendation: If pause-controller replacement is unnecessary, make each reference immutable and remove the corresponding setters; ownership of the controllers can still be transferred when emergency signers change.

If replacement must remain possible, introduce an immutable shared pause coordinator that maintains the canonical main and fast-tier pause states or controller addresses. Each consumer should reference this coordinator, with the channel's tier fixed at deployment, so a controller rotation is performed once rather than through several independent updates.

SOLV-6
Events Report the Quoted NAV Instead of the Execution NAV

Informational · Fixed

> **Update**
> `BTCPlusMainnetChannel.Minted` and `BTCPlusDepositor.Deposited` now emit the real-time NAV used for the conversion. Addressed in `20349bf`. Related documentation was updated in `699db1a`, `0bb7b0e`, and `d94e3e9`.

File(s) affected: `src/BTCPlusDepositor.sol`, `src/BTCPlusMainnetChannel.sol`

Description: The verifier-supplied `nav_` is used only to check whether the quote is within `MAX_NAV_DEVIATION_BPS` of the realtime NAV. `BTCPlusDepositor` calculates the number of BTCPlus shares through `getSharesByValue(solvBTCAmount_)`, which uses the realtime NAV at execution.

Both `BTCPlusMainnetChannel.Minted` and `BTCPlusDepositor.Deposited` emit the quoted `nav_`, but neither event emits the realtime NAV used for the conversion. The quoted and realtime values may differ by up to the permitted deviation. An indexer that interprets the event's `nav` field as the conversion price may therefore record an incorrect NAV for the mint.

Recommendation: Rename the existing event field to `quotedNav` and include the realtime NAV used for the share conversion in the emitted data.

SOLV-7

Registry Documentation Understates the Owner's Indirect Write Authority

Informational | Fixed

> **Update**
> The documentation has been corrected in `b9f13d3`, `b5d5d00`, and `0bb7b0e`.

File(s) affected: `src/BTCTxRegistry.sol`, `docs/SECURITY-SELF-REVIEW.md`, `docs/IMPLEMENTATION-SPEC.md`, `docs/MainnetMint-Architecture.md`, `README.md`

Description: The owner has no function that directly modifies `txReceivers`. However, it can authorize any non-zero address through `setChannel()`, after which that address can call `consume()` with arbitrary transaction hashes and receivers. Several comments and documents describe this more strongly than the implementation guarantees:

1. `src/BTCTxRegistry.sol:34-36` states that the owner has "no way to mark a hash spent." The owner cannot do so directly, but it can authorize itself or another address that can then call `consume()`.
2. `docs/SECURITY-SELF-REVIEW.md:55` states that the owner has "no room even to misconfigure" the registry and that the ledger grows only through a real mint. An incorrectly authorized address can consume arbitrary hashes without performing a mint because the registry does not verify any minting activity.
3. `docs/SECURITY-SELF-REVIEW.md:199` describes the owner as having "no write path." This should be qualified as no *direct* record-management path.
4. `docs/SECURITY-SELF-REVIEW.md:240` and `README.md:61` describe the registry tests as proving that the owner has no write path. The cited behavior only proves that the owner is not authorized by default; the owner can authorize itself through `setChannel()`.
5. `docs/IMPLEMENTATION-SPEC.md:138-140` states that the ledger grows only through an authorized channel "completing a real mint." The registry only verifies that `msg.sender` is authorized and does not establish that a mint occurred.
6. `docs/MainnetMint-Architecture.md:668` similarly states that the owner has no write path into the ledger without accounting for its control over the authorized-channel set.

This is not privilege escalation because channel authorization is an intended governance capability and a malicious owner is outside the stated threat model. The current wording nevertheless obscures the fact that channel authorization is the registry's primary trust boundary.

Recommendation: Replace the identified statements with language explaining that the owner has no *direct* record-import, modification, or deletion function, but controls which addresses may irreversibly write records through `consume()`. Clarify that the registry does not verify whether an authorized caller is a legitimate channel or whether consumption accompanies a real mint.

SOLV-8

Mint Signatures Are Transferable Between Authorized Minters

Informational | Fixed

> **Update**
> The development team added the minter's address to the corresponding signatures. Addressed in `f4b9450`. Related documentation was updated in `699db1a`, `0bb7b0e`, and `d94e3e9`.

File(s) affected: `src/BTCPlusMainnetChannel.sol`, `src/SolvBTCMainnetChannel.sol`, `docs/MainnetMint-Architecture.md`, `docs/SECURITY-SELF-REVIEW.md`

Description: The EIP-712 mint structures do not include the minter expected to submit the request. Authorization is checked separately by comparing `minterCaps[msg.sender]` with the signed amount. Consequently, any configured minter with a sufficient cap can submit a valid signature issued for the same channel, even if the verifier or operational system intended another minter to execute it.

A different authorized minter can copy a signature from operational infrastructure or pending transaction calldata, front-run the intended submission, choose when the request executes, and consume the associated replay record and shared channel allowance. The intended transaction would then revert as already processed. The submitting minter cannot change the signed receiver, amount, token, BTC transaction hash, expiry, or NAV, so this does not create a direct theft or unauthorized-mint path.

The documentation states that a compromised minter alone can do "nothing" because it lacks a verifier signature. More precisely, a minter cannot forge a new request, but it can execute any valid outstanding signature it obtains. This distinction is relevant to signature handling and incident response.

Recommendation: If each signature should be restricted to a particular submitter, include a `minter` address in both EIP-712 mint structures and require it to equal `msg.sender`. If signatures are intentionally executable by any authorized minter, document their transferable nature and clarify that a compromised minter may execute existing signatures but cannot create new ones.

SOLV-9

Reauthorized Channels Can Remain Blocked Beyond the Documented Recovery Window

Low | Fixed

> **Update**
> The usage is now reset one window after the latest configuration change. Addressed in `72b38ba93ddb7ec842d65ffe5330058e87ad65a4`.

File(s) affected: `src/utils/ChannelRateLimiter.sol`, `src/SolvBTCMintGateway.sol`, `src/BTCPlusDepositor.sol`

Description: The documentation for `_checkpoint()` states that the penalty following a long revocation is bounded by one window of the new configuration. However, when a channel is reauthorized with a lower allowance, its preserved usage can prevent minting for substantially longer. `_checkpoint()` retains the outstanding usage accumulated before revocation, while `_amountCanBeUsed()` subsequently decays it at the new, smaller allowance's rate. Usage does not decay during revocation, so even a channel that has been inactive for an extended period can remain blocked for many windows after reauthorization. This violates the documented recovery bound and can unexpectedly suspend gateway minting or depositor operations following a legitimate governance action.

Exploit Scenario:

1. A channel is configured with a per-transaction limit of `50` BTC, a window allowance of `100` BTC, and a window duration of `1` day.
2. The channel performs two mints of `50` BTC and is revoked within the same block, preserving `100` BTC of outstanding usage.
3. After `365` days, the owner reauthorizes the channel with a per-transaction limit of `0.5` BTC, a window allowance of `1` BTC, and a window duration of `1` day.
4. After one newly configured window, outstanding usage remains at `99` BTC. All mints remain blocked, despite the documented recovery period having elapsed.
5. At day `99` after reauthorization, outstanding usage still equals the `1` BTC allowance, so even a one-wei mint fails. Full allowance recovery takes `100` days, assuming no further configuration changes or minting.

Recommendation: Ensure that retained usage recovers within the documented window following reauthorization, including when the allowance is reduced, while preserving protection against allowance-reset bypasses. Add regression tests covering reauthorization with substantially lower allowances.

Auditor Suggestions

S1 `BTCTxRegistry` Does Not Expose the Consuming Channel

Fixed

> **Update**
> The development team implemented the suggestion by exposing a new `consumptions()` function. Addressed in `c53cdb0`. Related documentation was updated in `699db1a`, `0bb7b0e`, and `d94e3e9`.

File(s) affected: `src/BTCTxRegistry.sol`

Description: Both mainnet channels expose `txReceivers(btcTxHash_)`, which forwards directly to the shared `BTCTxRegistry`. Since the registry is shared, the getter returns a receiver when any authorized channel has consumed the hash.

For example, after a BTCPlus channel consumes a transaction hash, calling `txReceivers()` on `SolvBTCMainnetChannel` returns the BTCPlus recipient. The registry stores only the receiver in contract storage. The consuming channel is available through the `Consumed` event, but there is no getter that lets an on-chain caller identify which channel consumed the hash.

Recommendation: Consider storing the consuming channel alongside the receiver in `BTCTxRegistry` and exposing both values through a getter.

S2 Code Improvements

Fixed

> **Update**
> The development team implemented all eight recommendations in `04dc081`, `c8d229b`, `5a19b9e`, `9333150`, `f4e4a6c`, `1f8a6fe`, `809b54f`, and `c2de094`. Related documentation was updated in `699db1a`, `0bb7b0e`, and `d94e3e9`.

File(s) affected: `src/SolvBTCMainnetChannel.sol`, `src/BTCPlusMainnetChannel.sol`, `src/SolvBTCMintGateway.sol`, `src/BTCPlusDepositor.sol`, `src/BTCTxRegistry.sol`, `src/PauseController.sol`, `src/utils/ChannelRateLimiter.sol`, `src/interfaces/ISolvBTCMintGateway.sol`, `src/interfaces/IBTCPlusDepositor.sol`, `src/interfaces/IBTCTxRegistry.sol`

Description:

1. `SolvBTCMintGateway.mint()` does not check whether `to_` is the zero address. Calls from the current channel contracts cannot pass a zero recipient because both channels reject it first. However, another address authorized directly through `SolvBTCMintGateway.setChannelRateLimit()` can call `mint()` without that channel-level check. The gateway currently relies on the

external SolvBTC implementation to reject the mint. Add a zero-address check to the gateway so every authorized caller is subject to the same validation.

2. `InvalidMintedAmount(uint256 mintedAmount)` is only raised when `mintedAmount == 0`, so its argument always reports zero. Remove the argument or replace it with useful conversion context such as the deposited amount and realtime NAV.
3. `NavDiffOutOfRange()` and `SolvBTCNotReceived()` do not report the values that caused the failure. Including the quoted NAV, realtime NAV, expected amount, and received amount would make failed transactions easier to diagnose. `SolvBTCMismatch()` could likewise report both conflicting token addresses.
4. Several custom errors use the spelling `UnChanged`, including `VerifierUnChanged`, `PauseControllerUnChanged`, `VaultUnChanged`, and `ChannelUnChanged`. Rename them to use `Unchanged` consistently if ABI compatibility is not required.
5. `BTCPlusDepositor._checkNavAndConvertToShares()` uses the literal `10000` as the basis-point denominator. Define a named constant such as `BPS_DENOMINATOR` to make the calculation self-documenting.
6. `PauseController.mintPaused()` and `burnPaused()` only return private state variables. Declaring the variables as `public override` would generate the same external getters and remove the two manual functions.
7. `SolvBTCMintGateway` and `BTCPlusDepositor` describe themselves as the sole holders of the SolvBTC and BTCPlus minter roles. Neither contract can enforce this because the external token contracts control their own minter assignments. Revise these comments to describe the intended deployment configuration rather than a contract invariant.
8. `SolvBTCMintGateway`, `BTCPlusDepositor`, and `BTCTxRegistry` describe configured callers as channels. Their configuration functions accept any nonzero address and do not verify that it is a channel contract. Revise the comments to distinguish intended channel contracts from arbitrary authorized callers.

Recommendation: Consider applying the suggested improvements.

S3 Remove Redundant Chain Id and Token Fields

Fixed

Update

The redundant `chainId` and `token` have been removed from the signatures in `8599332`. Related documentation was updated in `699db1a`, `0bb7b0e`, and `d94e3e9`.

File(s) affected: `src/BTCPlusMainnetChannel.sol`, `src/SolvBTCMainnetChannel.sol`

Description: Both channels accept `chainId_` through `mint()` and include it in `MINT_TYPEHASH`, even though the contract can read the current chain from `block.chainid` and OpenZeppelin's EIP-712 domain already binds signatures to that chain. The separate `token` field in `MINT_TYPEHASH` is also redundant because each channel's token is immutable and the domain's `verifyingContract` field binds the signature to that particular channel.

These duplications increase calldata and signature-generation complexity while creating additional opportunities for integrations to provide internally inconsistent values.

Recommendation: Use `block.chainid` directly and remove `chainId_` from the public `mint()` interface and `MINT_TYPEHASH`. Consider also removing the separate `token` field from the signed structure. Coordinate these changes with the off-chain signing infrastructure because changing the type hash will invalidate signatures created under the previous schema.

S4 Separate Channel Authorization Lifecycle Operations

Fixed

Update

Separate internal helpers have been introduced for revoking, authorizing or updating rate limits. Note that the new `ChannelRevoked` event is emitted even when an unconfigured or already-revoked channel is revoked, so the event may not represent a real lifecycle transition. Addressed in `a3fecdf`.

Update

`ChannelRevoked` is no longer emitted when revoking a channel that is already unauthorized. Addressed in `72b38ba93ddb7ec842d65ffe5330058e87ad65a4`.

File(s) affected: `src/utils/ChannelRateLimiter.sol`, `src/SolvBTCMintGateway.sol`, `src/BTCPlusDepositor.sol`

Description: `_setChannelRateLimit()` currently handles initial authorization, revocation, and parameter updates based on the supplied window and existing state. This makes important side effects less explicit. For example, revocation clears `amountUsed` because the general clamping operation applies a new `maxWindowAmount` of zero. Combining these distinct transitions makes the code harder to reason about and more fragile when extended.

Recommendation: Separate authorization, revocation, and rate-limit updates into dedicated internal functions with explicit preconditions and state changes. The existing external setter can remain as a dispatcher if preserving the current ABI is desirable. Alternatively, expose distinct external operations and events to make governance intent clearer. Retain shared helpers for common parameter validation.

S5 Use Transient Storage for Reentrancy Protection

Fixed

Update
`ReentrancyGuardTransient` has replaced the standard reentrancy guard in all recommended contracts in `84a4c39`. Related documentation was updated in `17cb7c8`.

File(s) affected: `src/BTCPlusMainnetChannel.sol`, `src/SolvBTCMainnetChannel.sol`, `src/SolvBTCMintGateway.sol`, `src/BTCPlusDepositor.sol`

Description: The guarded contracts currently use the storage-based `ReentrancyGuard`. BSC mainnet supports EIP-1153, the project already targets the Cancun EVM version, and OpenZeppelin's `ReentrancyGuardTransient` provides equivalent protection using cheaper transient storage. The existing benchmark indicates a saving of approximately `4705` gas per guarded call.

Recommendation: Replace the storage-based guards with `ReentrancyGuardTransient`. Document that the resulting contracts require an EIP-1153-compatible network and rerun the reentrancy tests after the change.

S6 Consider Packing the Rate-Limit State

Fixed

Update
The refactored `RateLimit` now uses two slots instead of five. Addressed in `8086bf7`. Related documentation was updated in `699db1a`, `0bb7b0e`, and `d94e3e9`.

File(s) affected: `src/utils/ChannelRateLimiter.sol`

Description: Each `RateLimit` currently contains five `uint256` fields and occupies five storage slots. Since `_consume()` reads most or all of these fields during every mint or deposit, packing them could reduce the number of cold storage loads. Smaller integer fields introduce additional masking, casting, and read-modify-write operations, however, so the benefit depends on the final implementation and should be measured rather than assumed.

An illustrative two-slot layout is:

```
struct RateLimit {
    uint128 amountUsed;
    uint128 maxSingleAmount;

    uint128 maxWindowAmount;
    uint48 lastUsedAt;
    uint32 window;
}
```

Wider time fields could also be used while retaining a two-slot layout.

Recommendation: Experiment with packing `RateLimit` and benchmark the production implementation against the existing layout. If adopted, use checked conversions and appropriate bounds for narrowed fields, and test all consumption and reconfiguration paths.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/solv-finance/Solv4.0`

- `390...591 ./src/BTCPlusDepositor.sol`
- `233...d39 ./src/BTCPlusMainnetChannel.sol`
- `be8...d6f ./src/BTCTxRegistry.sol`
- `62a...032 ./src/PauseController.sol`
- `534...5a4 ./src/SolvBTCMainnetChannel.sol`
- `e1d...52d ./src/SolvBTCMintGateway.sol`
- `5aa...09f ./src/interfaces/IBTCPlus.sol`
- `c8f...dcb ./src/interfaces/IBTCPlusDepositor.sol`
- `48c...375 ./src/interfaces/IBTCTxRegistry.sol`
- `9f3...559 ./src/interfaces/IPauseController.sol`
- `b4c...95d ./src/interfaces/ISolvBTC.sol`
- `d97...7c0 ./src/interfaces/ISolvBTCMintGateway.sol`
- `43b...936 ./src/utils/ChannelRateLimiter.sol`

Test Suite Results

The test suite was run using `forge test`. The coverage results were obtained using `forge coverage`. The branch coverage for the contracts under audit is excellent at `95.40%`. The test suite included unit tests, fork tests, and fuzzing.

Update After First Fix Review: Additional tests have been added, and the branch coverage has increased to `96.47%`.

```
Ran 6 tests for test/PauseTopology.t.sol:PauseTopologyTest
[PASS] test_Topology_APauseSurvivesASignerRotation() (gas: 47569)
[PASS] test_Topology_ConstructionRefusesAZeroController() (gas: 67628)
[PASS] test_Topology_RotatingTheEmergencySignersStillWorks() (gas: 59139)
[PASS] test_Topology_TheFastControllerStopsOnlyItsTier() (gas: 221199)
[PASS] test_Topology_TheMainControllerIsStillTheGlobalStop() (gas: 54629)
[PASS] test_Topology_TheReferenceIsFixedAtConstruction() (gas: 15514)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 9.17ms (1.68ms CPU time)

Ran 5 tests for test/ChannelLifecycle.t.sol:PackedRateLimitTest
[PASS] test_Packed_AnAbsurdWindowRevertsRatherThanTruncating() (gas: 62550)
[PASS] test_Packed_TheGetterIsUnchanged() (gas: 133359)
[PASS] test_Packed_TheHardCapRoundTrips() (gas: 135687)
[PASS] test_Packed_TheLongestAcceptableWindowStillWorks() (gas: 67769)
[PASS] test_Packed_TimestampsFarInTheFutureFit() (gas: 133078)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 9.20ms (1.70ms CPU time)

Ran 7 tests for test/RateLimitBurst.t.sol:RateLimitBurstTest
[PASS] test_Burst_ABucketIsFullFromConfiguration() (gas: 67406)
[PASS] test_Burst_AFullWindowAllowsTwiceTheAllowance() (gas: 202105)
[PASS] test_Burst_ConfiguringLowThenRaisingStillStartsFull() (gas: 76638)
[PASS] test_Burst_NoInstantEverExceedsTheAllowance() (gas: 140283)
[PASS] test_Burst_OneHundredFiftyWithinTwelveHours() (gas: 156951)
[PASS] test_Burst_TheCeilingIsReachedAtEveryHorizon() (gas: 332532)
[PASS] test_Burst_TheSustainedRateIsTheAllowancePerWindow() (gas: 331096)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 10.19ms (2.41ms CPU time)

Ran 44 tests for test/BTCPlusDepositor.t.sol:BTCPlusDepositorTest
[PASS] testFuzz_Deposit_InvariantI1(uint256,uint256,uint256) (runs: 256, μ: 152436, ~: 152620)
[PASS] test_Admin_OwnershipTransferIsTwoStep() (gas: 32946)
[PASS] test_Admin_RevertNonOwner() (gas: 20320)
```

```
[PASS] test_Admin_RevertRateLimitAboveHardCap() (gas: 19178)
[PASS] test_Admin_RevertSetSameVault() (gas: 17788)
[PASS] test_Admin_RevertSetZeroVault() (gas: 13645)
[PASS] test_Admin_RevertSingleNotBelowWindowAmount() (gas: 18186)
[PASS] test_Admin_SetVault() (gas: 151658)
[PASS] test_Constructor_RevertInvalidBTCPlus() (gas: 72856)
[PASS] test_Constructor_RevertInvalidPauseController() (gas: 97026)
[PASS] test_Constructor_RevertInvalidSolvBTC() (gas: 72867)
[PASS] test_Constructor_RevertInvalidVault() (gas: 73041)
[PASS] test_Deposit_AtSingleAmountBoundary() (gas: 133406)
[PASS] test_Deposit_ChannelsAreMeteredIndependently() (gas: 358863)
[PASS] test_Deposit_DepositorHoldsNoBalance() (gas: 138723)
[PASS] test_Deposit_EventCarriesBothTheQuotedAndTheExecutionNav() (gas: 138036)
[PASS] test_Deposit_MintsToReceiverNotToCaller() (gas: 134723)
[PASS] test_Deposit_NavAtLowerBandBoundary() (gas: 133114)
[PASS] test_Deposit_NavAtUpperBandBoundary() (gas: 133081)
[PASS] test_Deposit_QuotedNavDoesNotAffectMintedAmount() (gas: 154157)
[PASS] test_Deposit_RateLimitChangeCheckpointsAtOldDecayRate() (gas: 151029)
[PASS] test_Deposit_RevertAfterAuthorisationRevoked() (gas: 36568)
[PASS] test_Deposit_RevertNavOneWeiAboveBand() (gas: 35161)
[PASS] test_Deposit_RevertNavOneWeiBelowBand() (gas: 35155)
[PASS] test_Deposit_RevertOnReentrancyFromVault() (gas: 2310676)
[PASS] test_Deposit_RevertSingleAmountExceeded() (gas: 24008)
[PASS] test_Deposit_RevertUnauthorisedChannel() (gas: 82418)
[PASS] test_Deposit_RevertWhenOracleReportsZeroNav() (gas: 49406)
[PASS] test_Deposit_RevertWhenPaused() (gas: 41681)
[PASS] test_Deposit_RevertWhenSharesRoundToZero() (gas: 47349)
[PASS] test_Deposit_RevertWhenVaultReceivesLessThanDeposited() (gas: 2184761)
[PASS] test_Deposit_RevertWindowAmountExceeded() (gas: 225901)
[PASS] test_Deposit_RevertZeroAmount() (gas: 19212)
[PASS] test_Deposit_RevertZeroNav() (gas: 19212)
[PASS] test_Deposit_RevertZeroReceiver() (gas: 17077)
[PASS] test_Deposit_RoundsDownInProtocolFavour() (gas: 131746)
[PASS] test_Deposit_SharesAreFewerThanValueWhenNavAboveOne() (gas: 128433)
[PASS] test_Deposit_StaleQuotedNavDoesNotDilute() (gas: 131921)
[PASS] test_Deposit_Success() (gas: 144356)
[PASS] test_Deposit_SurvivesDustCreditedToVault() (gas: 2175154)
[PASS] test_Deposit_SurvivesPreExistingVaultBalance() (gas: 146712)
[PASS] test_Deposit_ValueIsPreservedAcrossNavMove() (gas: 136093)
[PASS] test_Deposit_WindowRecoversLinearly() (gas: 244871)
[PASS] test_Initialization() (gas: 26122)
Suite result: ok. 44 passed; 0 failed; 0 skipped; finished in 42.15ms (35.30ms CPU time)

Ran 11 tests for test/Handover.t.sol:HandoverTest
[PASS] test_Handover_FailsWhenASingleChannelWasMissed() (gas: 9906764)
[PASS] test_Handover_FailsWhileTheTransferIsStillPending() (gas: 9968348)
[PASS] test_Handover_OnlyVerifyHandoverCatchesAnOutboundTransferFromTheRightOwner() (gas: 10072991)
[PASS] test_Handover_PassesOnceEveryAcceptanceHasLanded() (gas: 9908461)
[PASS] test_Handover_ThePauseControllersAnswerToTheEmergencyOwner() (gas: 9907459)
[PASS] test_Handover_TheReadOnlyVerifierAlsoRejectsAnUnacceptedHandover() (gas: 9995846)
[PASS] test_Preflight_AContractOwnerIsAccepted() (gas: 9865481)
[PASS] test_Preflight_ARehearsalThatKeepsTheSystemIsFine() (gas: 9629761)
[PASS] test_Preflight_RefusesBeforeAnythingIsCreated() (gas: 366193)
[PASS] test_Preflight_RefusesToHandTheSystemToAPlainAddress() (gas: 365603)
[PASS] test_Preflight_TheUpgradeScriptChecksItAsWell() (gas: 10017095)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 55.12ms (66.96ms CPU time)

Ran 36 tests for test/SolvBTCMintGateway.t.sol:SolvBTCMintGatewayTest
[PASS] testFuzz_RateLimit_NeverMintsMoreThanDecayAllows(uint256[8],uint256[8]) (runs: 256, μ: 172383, ~:
173980)
[PASS] testFuzz_RateLimit_OutstandingNeverExceedsWindowAmount(uint256,uint256) (runs: 256, μ: 86565, ~:
86352)
[PASS] test_Admin_FirstConfigurationStartsUnused() (gas: 68180)
[PASS] test_Admin_OwnershipTransferIsTwoStep() (gas: 33733)
[PASS] test_Admin_RaisingAllowanceKeepsOutstandingUsage() (gas: 99069)
[PASS] test_Admin_RateLimitChangeCheckpointsAtOldDecayRate() (gas: 104183)
[PASS] test_Admin_RateLimitChangeKeepsOutstandingAboveTheNewCap() (gas: 135903)
[PASS] test_Admin_RevertAboveHardCap() (gas: 19174)
[PASS] test_Admin_RevertNonOwner() (gas: 16317)
[PASS] test_Admin_RevertRevokeWithResidualAmounts() (gas: 17777)
[PASS] test_Admin_RevertSetRateLimitForZeroAddress() (gas: 13790)
[PASS] test_Admin_RevertWhenSingleAboveWindowAmount() (gas: 18249)
```

```
[PASS] test_Admin_RevertWhenSingleEqualsWindowAmount() (gas: 18318)
[PASS] test_Admin_RevokeChannel() (gas: 39491)
[PASS] test_Admin_SetAtHardCapBoundary() (gas: 68560)
[PASS] test_Admin_SetChannelRateLimit() (gas: 70491)
[PASS] test_Constructor_RevertInvalidPauseController() (gas: 67738)
[PASS] test_Constructor_RevertInvalidSolvBTC() (gas: 67643)
[PASS] test_Initialization() (gas: 16949)
[PASS] test_Mint_AtSingleAmountBoundary() (gas: 84286)
[PASS] test_Mint_ChannelIsNotItselfASolvBTCMinter() (gas: 17140)
[PASS] test_Mint_ChannelsAreMeteredIndependently() (gas: 196661)
[PASS] test_Mint_ResumesAfterUnpause() (gas: 88261)
[PASS] test_Mint_RevertChannelNotConfigured() (gas: 21436)
[PASS] test_Mint_RevertOnReentrancy() (gas: 1826799)
[PASS] test_Mint_RevertSingleAmountExceededByOneWei() (gas: 23785)
[PASS] test_Mint_RevertWhenPaused() (gas: 41579)
[PASS] test_Mint_RevertWindowAmountExceeded() (gas: 125190)
[PASS] test_Mint_RevertZeroReceiver() (gas: 16956)
[PASS] test_Mint_Success() (gas: 89914)
[PASS] test_Mint_WindowAccumulatesAcrossCalls() (gas: 126470)
[PASS] test_RateLimit_DecayDoesNotOvershootTheCap() (gas: 87370)
[PASS] test_RateLimit_DecaysLinearlyToTheWei() (gas: 88902)
[PASS] test_RateLimit_FullyRecoversAfterOneWindow() (gas: 125514)
[PASS] test_RateLimit_PartialRecoveryIsSpendableToTheWei() (gas: 141287)
[PASS] test_RemainingMintAmount_IsZeroForUnconfiguredChannel() (gas: 10031)
Suite result: ok. 36 passed; 0 failed; 0 skipped; finished in 53.01ms (54.80ms CPU time)

Ran 10 tests for test/MinWindow.t.sol:MinWindowTest
[PASS] test_MinWindow_LongWindowsAreUnaffected() (gas: 67724)
[PASS] test_MinWindow_PreflightAcceptsTheFloor() (gas: 30744)
[PASS] test_MinWindow_PreflightRejectsAShortWindow() (gas: 30814)
[PASS] test_MinWindow_PreflightRejectsMismatchedOracleDecimals() (gas: 843101)
[PASS] test_MinWindow_RevocationIsStillPossible() (gas: 57544)
[PASS] test_MinWindow_TheBoundaryIsExact() (gas: 75093)
[PASS] test_MinWindow_TheDepositorRejectsItToo() (gas: 18222)
[PASS] test_MinWindow_TheGatewayRejectsADroppedDigit() (gas: 18342)
[PASS] test_MinWindow_TheGatewayRejectsAOneSecondWindow() (gas: 18275)
[PASS] test_MinWindow_TheScriptMirrorMatchesTheContract() (gas: 16856)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 1.29ms (721.96µs CPU time)

Ran 18 tests for test/MainnetChannels.integration.t.sol:MainnetChannelsIntegrationTest
[PASS] test_Metering_ExhaustingBTCPlusChannelDoesNotStarveSolvBTC() (gas: 2066144)
[PASS] test_Metering_ExhaustingSolvBTCChannelDoesNotStarveBTCPlus() (gas: 1950805)
[PASS] test_Metering_RevokingOneChannelLeavesTheOther() (gas: 1614417)
[PASS] test_Metering_TotalIssuanceScalesWithChannelCount() (gas: 2362897)
[PASS] test_Pause_OneSwitchStopsBothChannels() (gas: 1367302)
[PASS] test_Pause_OnlyEmergencyMultisigCanPause() (gas: 16574)
[PASS] test_Pause_UnpauseRestoresBothChannels() (gas: 1660015)
[PASS] test_Signatures_CannotCrossBetweenChannels() (gas: 1335201)
[PASS] test_TwoTier_ExhaustingTheFastWindowLeavesTheManualTierFull() (gas: 2820766)
[PASS] test_TwoTier_FastChannelIsCappedWellBelowTheManualOne() (gas: 2449972)
[PASS] test_TwoTier_GatewayControllerIsStillTheGlobalKillSwitch() (gas: 2204647)
[PASS] test_TwoTier_ManualTierAcceptsASafeVerifier() (gas: 4217863)
[PASS] test_TwoTier_OneDepositCannotBeMintedThroughBothTiers() (gas: 2322077)
[PASS] test_TwoTier_PausingTheFastTierLeavesTheManualTierRunning() (gas: 2360299)
[PASS] test_TwoTier_RaisingTheBotCapDoesNotRaiseTheChannelCeiling() (gas: 2164900)
[PASS] test_WaveOne_OnlyBTCPlusChannelIsAuthorised() (gas: 272005)
[PASS] test_WaveTwo_AddsChannelWithoutRedeployingAnything() (gas: 1660395)
[PASS] test_WaveTwo_DoesNotResetExistingChannelUsage() (gas: 1555360)
Suite result: ok. 18 passed; 0 failed; 0 skipped; finished in 15.08ms (12.06ms CPU time)

Ran 44 tests for test/SolvBTCMainnetChannel.t.sol:SolvBTCMainnetChannelTest
[PASS] testFuzz_Mint_AlwaysOneToOne(uint256) (runs: 256, µ: 166707, ~: 166394)
[PASS] test_Admin_OwnershipTransferIsTwoStep() (gas: 33920)
[PASS] test_Admin_RevertNonOwner() (gas: 20165)
[PASS] test_Admin_RevertSetInvalidVerifier() (gas: 13613)
[PASS] test_Admin_RevertSetMinterCapForVerifier() (gas: 17558)
[PASS] test_Admin_RevertSetMinterCapForZeroAddress() (gas: 13607)
[PASS] test_Admin_RevertSetSameVerifier() (gas: 17538)
[PASS] test_Admin_RevertSetVerifierToMinter() (gas: 19745)
[PASS] test_Admin_SetMinterCap() (gas: 27299)
[PASS] test_Admin_SetVerifier() (gas: 28419)
[PASS] test_Constructor_RevertInvalidGateway() (gas: 73833)
```

```
[PASS] test_Constructor_RevertInvalidPauseController() (gas: 103317)
[PASS] test_Constructor_RevertInvalidVerifier() (gas: 77256)
[PASS] test_Constructor_RevertWhenGatewayReportsZeroSolvBTC() (gas: 77897)
[PASS] test_Gateway_RateLimitExhaustionIsRetryable() (gas: 603301)
[PASS] test_Gateway_RevertSingleAmountExceeded() (gas: 102656)
[PASS] test_Gateway_RevertWhenAuthorisationRevoked() (gas: 129190)
[PASS] test_Gateway_RevertWhenGatewayLacksMinterRole() (gas: 113591)
[PASS] test_Initialization() (gas: 30639)
[PASS] test_Initialization_Eip712Domain() (gas: 12493)
[PASS] test_Mint_ChannelHoldsNoMintingRights() (gas: 17111)
[PASS] test_Mint_DeliversDirectlyWithoutTouchingTheChannel() (gas: 172545)
[PASS] test_Mint_ExpireTimeAtMaxValidityBoundary() (gas: 166101)
[PASS] test_Mint_IsOneToOne() (gas: 169615)
[PASS] test_Mint_RevertDoubleMint() (gas: 171624)
[PASS] test_Mint_RevertExceedCap() (gas: 30597)
[PASS] test_Mint_RevertExpireTimeTooFarOut() (gas: 37212)
[PASS] test_Mint_RevertExpiredSignature() (gas: 36686)
[PASS] test_Mint_RevertInvalidAmount() (gas: 24486)
[PASS] test_Mint_RevertInvalidBtcTxHash() (gas: 24482)
[PASS] test_Mint_RevertInvalidOpReturn() (gas: 29008)
[PASS] test_Mint_RevertInvalidReceiver() (gas: 22413)
[PASS] test_Mint_RevertInvalidSignature() (gas: 39963)
[PASS] test_Mint_RevertOpReturnForAnotherChain() (gas: 31137)
[PASS] test_Mint_RevertWhenAmountTampered() (gas: 45944)
[PASS] test_Mint_RevertWhenCallerIsNotMinter() (gas: 30627)
[PASS] test_Mint_RevertWhenOpReturnEncodesAnotherReceiver() (gas: 32455)
[PASS] test_Mint_RevertWhenPaused() (gas: 50923)
[PASS] test_Mint_RevertWhenSignedByNonVerifier() (gas: 43745)
[PASS] test_Mint_RevertWhenSignedForTheBaselineDomain() (gas: 44192)
[PASS] test_Mint_Success() (gas: 173236)
[PASS] test_Security_LeakedMinterCannotMintWithoutSignature() (gas: 36058)
[PASS] test_Security_LeakedVerifierCannotMintWithoutMinterCap() (gas: 31859)
[PASS] test_Security_RevertOnReentrancy() (gas: 3554867)
Suite result: ok. 44 passed; 0 failed; 0 skipped; finished in 62.57ms (68.13ms CPU time)

Ran 10 tests for test/RecordPromotion.t.sol:RecordPromotionTest
[PASS] test_Paths_PendingSitsBesideTheLiveRecord() (gas: 9690854)
[PASS] test_Pending_AStalePendingFileIsReplacedNotMergedInto() (gas: 19200570)
[PASS] test_Pending_AnUnconfirmedRunLeavesTheLiveRecordByteForByte() (gas: 19207800)
[PASS] test_Pending_AnUnconfirmedUpgradeArchivesNothing() (gas: 20567560)
[PASS] test_Pending_TheAddressesGoToAPendingFile() (gas: 19206777)
[PASS] test_Promote_DoesNotArchiveWhenTheGatewayDidNotChange() (gas: 10133620)
[PASS] test_Promote_MakesThePendingRecordLiveAndArchivesWhatItReplaced() (gas: 21017375)
[PASS] test_Promote_RefusesAPendingRecordWhoseContractsWereNeverDeployed() (gas: 19323959)
[PASS] test_Promote_RefusesWhenTheChainDoesNotMatchTheConfig() (gas: 19380792)
[PASS] test_Promote_RefusesWhenThereIsNothingPending() (gas: 9705550)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 69.09ms (103.48ms CPU time)

Ran 18 tests for test/BTCTxRegistry.t.sol:BTCTxRegistryTest
[PASS] testFuzz_FirstConsumeWinsRegardlessOfChannel(bytes32,address,address,bool) (runs: 256, μ: 68619,
~: 68628)
[PASS] test_Consume_AnUnconsumedHashIsEmptyInBothFields() (gas: 10209)
[PASS] test_Consume_DistinctHashesAreIndependent() (gas: 118465)
[PASS] test_Consume_OwnerCanSelfAuthoriseAndBurnAHash() (gas: 98124)
[PASS] test_Consume_OwnerIsNotAuthorisedByDefault() (gas: 15835)
[PASS] test_Consume_RecordsTheReceiver() (gas: 65410)
[PASS] test_Consume_RecordsWhichChannelConsumedTheHash() (gas: 64845)
[PASS] test_Consume_RevertSameChannelTwice() (gas: 64377)
[PASS] test_Consume_RevertSecondChannelCannotSpendTheSameDeposit() (gas: 71880)
[PASS] test_Consume_RevertUnauthorisedCaller() (gas: 15823)
[PASS] test_Consume_RevertZeroHash() (gas: 15717)
[PASS] test_Consume_RevertZeroReceiver() (gas: 13619)
[PASS] test_Initialization() (gas: 18377)
[PASS] test_SetChannel_EmitsAndRevokes() (gas: 24798)
[PASS] test_SetChannel_OnlyOwner() (gas: 13889)
[PASS] test_SetChannel_RevertUnchanged() (gas: 18220)
[PASS] test_SetChannel_RevertZeroAddress() (gas: 13706)
[PASS] test_SetChannel_RevocationDoesNotReleaseRecords() (gas: 75663)
Suite result: ok. 18 passed; 0 failed; 0 skipped; finished in 8.34ms (8.08ms CPU time)

Ran 10 tests for test/ChannelLifecycle.t.sol:ChannelLifecycleTest
[PASS] test_Lifecycle_AuthorisingAnnouncesItself() (gas: 71642)
```

```
[PASS] test_Lifecycle_DispatchFollowsTheChannelState() (gas: 78509)
[PASS] test_Lifecycle_ReauthorisingIsAGrant() (gas: 85910)
[PASS] test_Lifecycle_RevocationRefusesLeftoverAmounts() (gas: 68585)
[PASS] test_Lifecycle_RevokingAnUnconfiguredChannelIsNotATransition() (gas: 26499)
[PASS] test_Lifecycle_RevokingAnnouncesItself() (gas: 60432)
[PASS] test_Lifecycle_RevokingTwiceAnnouncesOnce() (gas: 132615)
[PASS] test_Lifecycle_TheSplitChangedNoBehaviour() (gas: 162444)
[PASS] test_Lifecycle_UpdatingIsNotAGrant() (gas: 76123)
[PASS] test_Lifecycle_ValidationAppliesToUpdatesToo() (gas: 72108)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 2.13ms (2.89ms CPU time)

Ran 13 tests for test/DeployTestnet.t.sol:DeployTestnetTest
[PASS] test_Deploy_AllChannelsShareOneRegistry() (gas: 9821997)
[PASS] test_Deploy_DefersRoleGrantsWhenDeployerIsNotTokenAdmin() (gas: 10014973)
[PASS] test_Deploy_DepositorIsConfiguredOnlyForBTCPlusChannels() (gas: 9639123)
[PASS] test_Deploy_EndToEndAndThenMints() (gas: 9898157)
[PASS] test_Deploy_HandoverLeavesOwnershipPending() (gas: 9925557)
[PASS] test_Deploy_PreflightRejectsEmergencyOwnerEqualToOwner() (gas: 365496)
[PASS] test_Deploy_PreflightRejectsMinterCapAboveRateLimit() (gas: 1335193)
[PASS] test_Deploy_PreflightRejectsWindowAboveHardCap() (gas: 355011)
[PASS] test_Deploy_PreflightRejectsWrongChainId() (gas: 339936)
[PASS] test_Deploy_ReusedRegistryStillKnowsSpentDeposits() (gas: 32414105)
[PASS] test_Deploy_SharedEmergencyOwnerIsAllowedWhenWaived() (gas: 9887569)
[PASS] test_Deploy_SharedEmergencyOwnerRejectedWhenOwnerIsATimelock() (gas: 443158)
[PASS] test_Deploy_TiersAreSeparated() (gas: 9802578)
Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 46.20ms (40.89ms CPU time)

Ran 69 tests for test/BTCPlusMainnetChannel.t.sol:BTCPlusMainnetChannelTest
[PASS] testFuzz_Mint_NeverMintsMoreValueThanDeposited(uint256,uint256) (runs: 256, μ: 263524, ~: 263763)
[PASS] test_Admin_RevertNonOwner() (gas: 26516)
[PASS] test_Admin_RevertSetInvalidVerifier() (gas: 13546)
[PASS] test_Admin_RevertSetMinterCapForVerifier() (gas: 17657)
[PASS] test_Admin_RevertSetMinterCapForZeroAddress() (gas: 13746)
[PASS] test_Admin_RevertSetSameVerifier() (gas: 17538)
[PASS] test_Admin_RevertSetVerifierToMinter() (gas: 19744)
[PASS] test_Admin_SetMinterCap() (gas: 27538)
[PASS] test_Admin_SetVerifier() (gas: 28330)
[PASS] test_Constructor_RevertInvalidDepositor() (gas: 76753)
[PASS] test_Constructor_RevertInvalidGateway() (gas: 76739)
[PASS] test_Constructor_RevertInvalidPauseController() (gas: 110342)
[PASS] test_Constructor_RevertInvalidVerifier() (gas: 84258)
[PASS] test_Constructor_RevertOnSolvBTCMismatch() (gas: 726896)
[PASS] test_Constructor_RevertWhenDepositorReportsZeroBTCPlus() (gas: 84129)
[PASS] test_Constructor_RevertWhenGatewayReportsZeroSolvBTC() (gas: 80893)
[PASS] test_Dust_BTCPlusSentToReceiverDoesNotBlockMinting() (gas: 3996844)
[PASS] test_Dust_SolvBTCSentToChannelDoesNotBlockMinting() (gas: 288847)
[PASS] test_Initialization() (gas: 36782)
[PASS] test_Initialization_Eip712Domain() (gas: 12516)
[PASS] test_Mint_ExpireTimeAtMaxValidityBoundary() (gas: 262706)
[PASS] test_Mint_NavAtLowerBandBoundary() (gas: 262083)
[PASS] test_Mint_NavAtUpperBandBoundary() (gas: 262049)
[PASS] test_Mint_QuotedNavDoesNotAffectMintedAmount() (gas: 366698)
[PASS] test_Mint_RevertDoubleMint() (gas: 266891)
[PASS] test_Mint_RevertExceedCap() (gas: 30692)
[PASS] test_Mint_RevertExpireTimeTooFarOut() (gas: 37327)
[PASS] test_Mint_RevertExpiredSignature() (gas: 36846)
[PASS] test_Mint_RevertInvalidAmount() (gas: 24593)
[PASS] test_Mint_RevertInvalidBtcTxHash() (gas: 24698)
[PASS] test_Mint_RevertInvalidOpReturn() (gas: 29175)
[PASS] test_Mint_RevertInvalidReceiver() (gas: 22519)
[PASS] test_Mint_RevertInvalidSignature() (gas: 40179)
[PASS] test_Mint_RevertNavAboveBand() (gas: 208425)
[PASS] test_Mint_RevertNavBelowBand() (gas: 208440)
[PASS] test_Mint_RevertOpReturnForAnotherChain() (gas: 31370)
[PASS] test_Mint_RevertWhenCallerIsNotMinter() (gas: 30700)
[PASS] test_Mint_RevertWhenOpReturnEncodesAnotherReceiver() (gas: 32645)
[PASS] test_Mint_RevertWhenOracleReportsZeroNav() (gas: 224010)
[PASS] test_Mint_RevertWhenPaused() (gas: 51027)
[PASS] test_Mint_RevertWhenQuotedNavTampered() (gas: 46369)
[PASS] test_Mint_RevertWhenSharesRoundToZero() (gas: 227151)
[PASS] test_Mint_RevertWhenSignedByNonVerifier() (gas: 44105)
[PASS] test_Mint_RevertWhenSignedForTheBaselineDomain() (gas: 44449)
```

```
[PASS] test_Mint_RevertZeroNav() (gas: 30723)
[PASS] test_Mint_RoundsDownInProtocolFavour() (gas: 261995)
[PASS] test_Mint_SharesAreFewerThanValueWhenNavAboveOne() (gas: 261852)
[PASS] test_Mint_StaleQuotedNavDoesNotDilute() (gas: 262138)
[PASS] test_Mint_Success() (gas: 270712)
[PASS] test_Mint_ValueIsPreservedAcrossNavMove() (gas: 266288)
[PASS] test_Pause_OneSwitchStopsAllThreeLayers() (gas: 4098734)
[PASS] test_Rescue_NoopOnEmptyBalance() (gas: 29382)
[PASS] test_Rescue_RecoversStrandedSolvBTC() (gas: 78617)
[PASS] test_Rescue_RevertZeroRecipient() (gas: 15808)
[PASS] test_Security_LeakedMinterCannotMintWithoutSignature() (gas: 36341)
[PASS] test_Security_LeakedVerifierCannotMintWithoutMinterCap() (gas: 31998)
[PASS] test_Security_RevertOnReentrancyFromVault() (gas: 5361974)
[PASS] test_TwoLeg_ChannelHoldsNoMintingRights() (gas: 17948)
[PASS] test_TwoLeg_ChannelsAreMeteredIndependently() (gas: 2456816)
[PASS] test_TwoLeg_ChargesBothRateLimits() (gas: 267683)
[PASS] test_TwoLeg_DepositorLimitCanBindFirst() (gas: 392877)
[PASS] test_TwoLeg_LeavesNoResidualAllowance() (gas: 264583)
[PASS] test_TwoLeg_RateLimitExhaustionIsRetryable() (gas: 864140)
[PASS] test_TwoLeg_RevertWhenDepositorAuthorisationRevoked() (gas: 225803)
[PASS] test_TwoLeg_RevertWhenDepositorLacksBTCPlusMinterRole() (gas: 245108)
[PASS] test_TwoLeg_RevertWhenGatewayAuthorisationRevoked() (gas: 137116)
[PASS] test_TwoLeg_RevertWhenGatewayLacksSolvBTCMinterRole() (gas: 119062)
[PASS] test_TwoLeg_RevertWhenLegOneUnderDelivers() (gas: 1938599)
[PASS] test_TwoLeg_SettlesThroughGatewayAndDepositor() (gas: 272912)
Suite result: ok. 69 passed; 0 failed; 0 skipped; finished in 95.08ms (109.03ms CPU time)

Ran 7 tests for test/DeployMainnet.t.sol:DeployMainnetTest
[PASS] test_Mainnet_DeployerNeverOwnsAnything() (gas: 8793389)
[PASS] test_Mainnet_ExecutingThePlanProducesAVerifiedSystem() (gas: 13636035)
[PASS] test_Mainnet_NothingIsAuthorisedBeforeTheSafeExecutes() (gas: 8796941)
[PASS] test_Mainnet_PlanDefaultFollowsThePendingRecord() (gas: 26353804)
[PASS] test_Mainnet_PlanIsEmptyOnceEverythingHasBeenExecuted() (gas: 27009920)
[PASS] test_Mainnet_PreflightRejectsEOAOwner() (gas: 206038)
[PASS] test_Mainnet_TheSafeBatchOnDiskIsThePlan() (gas: 13920365)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 89.86ms (106.61ms CPU time)

Ran 10 tests for test/WaveTwo.t.sol:WaveTwoTest
[PASS] test_WaveTwo_AddsOnlyTheChannels() (gas: 10172180)
[PASS] test_WaveTwo_BothWavesMintAfterwards() (gas: 10558111)
[PASS] test_WaveTwo_DoesNotCreateASecondMinter() (gas: 10266271)
[PASS] test_WaveTwo_DoesNotReauthoriseWaveOnesChannels() (gas: 10215517)
[PASS] test_WaveTwo_LeavesWaveOnesUsageAlone() (gas: 10464281)
[PASS] test_WaveTwo_MainnetPlanContainsOnlyTheNewWork() (gas: 41267904)
[PASS] test_WaveTwo_PassesStandaloneVerification() (gas: 10389202)
[PASS] test_WaveTwo_RefusesAGatewayForAnotherToken() (gas: 8780385)
[PASS] test_WaveTwo_RefusesToReuseTheGatewayWithAFreshPauseController() (gas: 7230738)
[PASS] test_WaveTwo_SharesTheReplayLedgerWithWaveOne() (gas: 10419547)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 33.84ms (55.45ms CPU time)

Ran 20 tests for test/UpgradeTestnet.t.sol:UpgradeTestnetTest
[PASS] test_Upgrade_AcceptsAConfigThatNamesTheLiveRegistry() (gas: 20441871)
[PASS] test_Upgrade_ArchivesTheSupersededDeployment() (gas: 20373159)
[PASS] test_Upgrade_CanBeRunTwice() (gas: 47953618)
[PASS] test_Upgrade_CodeIdentityDistinguishesContracts() (gas: 26608544)
[PASS] test_Upgrade_CodeIdentityReportsUnknownRatherThanGuessing() (gas: 16908137)
[PASS] test_Upgrade_HandsOverTheRevocationWhenTheSignerIsNotTokenAdmin() (gas: 21039399)
[PASS] test_Upgrade_OldChannelsAreDeauthorisedAndTheirAllowancesZeroed() (gas: 20377998)
[PASS] test_Upgrade_RefusesAChannelDroppedFromTheConfig() (gas: 10081075)
[PASS] test_Upgrade_RefusesAConfigThatNamesADifferentRegistry() (gas: 9805946)
[PASS] test_Upgrade_RefusesASenderThatDoesNotOwnTheRegistry() (gas: 10058498)
[PASS] test_Upgrade_RefusesWhenNothingChanged() (gas: 9995351)
[PASS] test_Upgrade_ReplacesTheContractsAndKeepsTheIrreplaceableOnes() (gas: 20373161)
[PASS] test_Upgrade_RetiresDroppedChannelsWhenAskedTo() (gas: 17907232)
[PASS] test_Upgrade_TheArchiveKeepsChannelsTheNewConfigDropped() (gas: 17867612)
[PASS] test_Upgrade_TheNewSystemMintsBothTokens() (gas: 20665457)
[PASS] test_Upgrade_TheOldChannelCanNoLongerMint() (gas: 20404345)
[PASS] test_Upgrade_TheOldGatewayAndDepositorLoseTheirMinterRoles() (gas: 20371980)
[PASS] test_Upgrade_TheOldGatewayRefusesTheOldChannelOnItsOwn() (gas: 20415303)
[PASS] test_Upgrade_TheReusedRegistryStillRefusesASpentDeposit() (gas: 20572746)
[PASS] test_Upgrade_ToleratesRetirementStepsAlreadyDone() (gas: 20032075)
Suite result: ok. 20 passed; 0 failed; 0 skipped; finished in 111.46ms (203.78ms CPU time)
```

```
Ran 40 tests for test/Security.t.sol:SecurityTest
[PASS] testFuzz_RateLimit_InvariantHoldsUnderAdminChurn(uint256[8]) (runs: 256, μ: 212792, ~: 215151)
[PASS] test_Assets_DonationToDepositorIsInert() (gas: 295938)
[PASS] test_Assets_DonationToGatewayIsInert() (gas: 193860)
[PASS] test_Assets_OneChannelsAllowanceCannotBeSpentByAnother() (gas: 202083)
[PASS] test_Audit_S1_DormantSignatureRejectedEarlyThenExecutable() (gas: 276999)
[PASS] test_Audit_S2_ConstructorRejectsTokenAsVault() (gas: 73283)
[PASS] test_Audit_S2_VaultCannotBeEitherToken() (gas: 22710)
[PASS] test_Audit_S4_GatewayRejectsZeroAmount() (gas: 19020)
[PASS] test_Gateway_MintToZeroAddressRevertsAtTheGateway() (gas: 75084)
[PASS] test_OpReturn_RejectsCrossTokenPayload() (gas: 31041)
[PASS] test_Pause_AdminFunctionsRemainAvailableWhilePaused() (gas: 159054)
[PASS] test_Pause_DoesNotInvalidateOutstandingSignatures() (gas: 173545)
[PASS] test_RateLimit_AnIdenticalResetRestartsTheExpiryClock() (gas: 147615)
[PASS] test_RateLimit_DecayStillAppliesBeforeTheExpiry() (gas: 100050)
[PASS] test_RateLimit_ExpiryIsMeasuredFromTheLastCheckpoint() (gas: 294566)
[PASS] test_RateLimit_LowerWaitRaiseYieldsNoMoreThanTheHigherConfigurationAlone() (gas: 249786)
[PASS] test_RateLimit_LoweringBelowOutstandingUsageWaitsOneWindowAtMost() (gas: 112440)
[PASS] test_RateLimit_MinterCapIsPerTransactionNotCumulative() (gas: 531718)
[PASS] test_RateLimit_ReauthorisationAtALowerAllowanceWaitsOneWindowAtMost() (gas: 171909)
[PASS] test_RateLimit_RevokeThenReauthorizeKeepsOutstandingUsage() (gas: 205397)
[PASS] test_RateLimit_UsageResumesDecayingAfterReauthorisation() (gas: 107495)
[PASS] test_Replay_ChannelWithoutRegistryAuthorisationCannotMint() (gas: 1695163)
[PASS] test_Replay_FailedMintLeavesTheDepositUnspent() (gas: 459254)
[PASS] test_Replay_OneDepositCannotBeMintedThroughBothChannels() (gas: 280251)
[PASS] test_Replay_OneDepositCannotBeMintedThroughBothChannels_MirrorDirection() (gas: 187905)
[PASS] test_Replay_RedeployedChannelInheritsSpentDeposits() (gas: 1942061)
[PASS] test_Replay_RegistryRevocationStopsOneChannelOnly() (gas: 210670)
[PASS] test_Sig_ASignatureCannotBeExecutedByADifferentMinter() (gas: 220629)
[PASS] test_Sig_BTCPlusSignatureCannotBeExecutedByADifferentMinter() (gas: 74959)
[PASS] test_Sig_CrossInstanceReplay_BTCPlusChannel() (gas: 1842257)
[PASS] test_Sig_CrossInstanceReplay_SolvBTCChannel() (gas: 1370233)
[PASS] test_Sig_ERC1271ContractVerifierSupported() (gas: 411418)
[PASS] test_Sig_ERC1271VerifierRejectedOnBadMagic() (gas: 308944)
[PASS] test_Sig_ExpireTimeAtCurrentTimestampIsValid() (gas: 164910)
[PASS] test_Sig_HighSMalleationRejected_BTCPlusChannel() (gas: 282275)
[PASS] test_Sig_HighSMalleationRejected_SolvBTCChannel() (gas: 42360)
[PASS] test_Sig_RotatedVerifierInvalidatesOldSignatures() (gas: 59730)
[PASS] test_Sig_VerifierAcquiringCodeBreaksECDSAVerification() (gas: 177576)
[PASS] test_Sig_ZeroedMinterCapDisablesMinter() (gas: 38192)
[PASS] test_Vault_CannotBeSetToDepositorItself() (gas: 13851)
Suite result: ok. 40 passed; 0 failed; 0 skipped; finished in 67.69ms (75.17ms CPU time)

Ran 15 tests for test/PauseController.t.sol:PauseControllerTest
[PASS] test_Auth_EveryStateChangeIsOwnerOnly() (gas: 28388)
[PASS] test_Auth_ReadingIsOpen() (gas: 39197)
[PASS] test_Bits_AreIndependent() (gas: 44359)
[PASS] test_Bits_BurnPauseSucceedsAndPersistsWhileStoppingNothing() (gas: 38694)
[PASS] test_Burn_PauseAndUnpause() (gas: 30267)
[PASS] test_Burn_PausingTwiceReverts() (gas: 37448)
[PASS] test_Burn_UnpausingWhenNotPausedReverts() (gas: 15182)
[PASS] test_Construction_OwnerIsSetOutrightAndNothingIsPaused() (gas: 15204)
[PASS] test_Construction_RejectsTheZeroOwner() (gas: 36285)
[PASS] test_Mint_PauseAndUnpause() (gas: 30286)
[PASS] test_Mint_PausingTwiceReverts() (gas: 38590)
[PASS] test_Mint_UnpausingWhenNotPausedReverts() (gas: 15116)
[PASS] test_Ownership_OnlyThePendingOwnerCanAccept() (gas: 42354)
[PASS] test_Ownership_TransferNeedsAcceptanceAndTheOldOwnerKeepsTheSwitch() (gas: 41981)
[PASS] test_Reuse_APauseInForceSurvivesEverythingExceptReplacement() (gas: 343187)
Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 142.98ms (2.40ms CPU time)

Ran 16 tests for test/UpdateLimits.t.sol:UpdateLimitsTest
[PASS] test_Rotate_ChangesTheVerifier() (gas: 11009877)
[PASS] test_Rotate_RevokesThePreviousMinter() (gas: 11228497)
[PASS] test_Rotate_SendsNothingWhenNobodyChanged() (gas: 15426992)
[PASS] test_Rotate_UpdatesTheDeploymentRecord() (gas: 17707313)
[PASS] test_Update_AppliesNewLimitsToGatewayAndDepositor() (gas: 13061839)
[PASS] test_Update_LeavesDepositorAloneForSolvBTCChannels() (gas: 13060833)
[PASS] test_Update_QueuesOnlyWhatDiffers() (gas: 12197237)
[PASS] test_Update_RejectsWindowBelowSingle() (gas: 10537756)
[PASS] test_Update_SendsNothingWhenAlreadyInSync() (gas: 10619269)
```

```
[PASS] test_Verify_FailsWhenAChannelHasTheWrongOwner() (gas: 9662108)
[PASS] test_Verify_FailsWhenBothTiersShareOnePauseController() (gas: 9644266)
[PASS] test_Verify_FailsWhenTheChainDriftsFromTheConfig() (gas: 9640087)
[PASS] test_Verify_FailsWhenTheDepositorIsPointedAtAnotherPauseController() (gas: 11221267)
[PASS] test_Verify_FailsWhenTheGatewayIsPointedAtAnotherPauseController() (gas: 9936555)
[PASS] test_Verify_FailsWhenTheReusedRegistryIsNotTheOneLive() (gas: 10013718)
[PASS] test_Verify_PassesOnAFreshDeployment() (gas: 9673344)
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in 144.41ms (264.47ms CPU time)

Ran 9 tests for test/SolvBTCMainnetChannel.fork.t.sol:SolvBTCMainnetChannelForkTest
[PASS] testFuzz_Fork_Mint_IsExactlyOneToOne(uint256) (runs: 256, μ: 172343, ~: 172048)
[PASS] test_Fork_Mint_EndToEnd() (gas: 185336)
[PASS] test_Fork_Mint_ProducesTransferableTokens() (gas: 202345)
[PASS] test_Fork_Mint_RevertDoubleMint() (gas: 170966)
[PASS] test_Fork_Mint_RevertExpireTimeTooFarOut() (gas: 35440)
[PASS] test_Fork_Mint_RevertWhenGatewayLacksMinterRole() (gas: 2171511)
[PASS] test_Fork_Mint_RevertWhenPaused() (gas: 49075)
[PASS] test_Fork_ProductionPreconditions() (gas: 38324)
[PASS] test_Fork_RateLimit_ExhaustionIsRetryable() (gas: 624421)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 2.00s (1.42s CPU time)

Ran 20 tests for test/BTCPlusMainnetChannel.fork.t.sol:BTCPlusMainnetChannelForkTest
[PASS] testFuzz_Fork_NeverMintsMoreValueThanDeposited(uint256,uint256) (runs: 256, μ: 347015, ~: 347195)
[PASS] test_Fork_Dust_BTCPlusSentToReceiverDoesNotBlockMinting() (gas: 520929)
[PASS] test_Fork_Dust_SolvBTCSentToChannelDoesNotBlockMinting() (gas: 410666)
[PASS] test_Fork_Mint_EndToEnd() (gas: 373113)
[PASS] test_Fork_Mint_FullValidityNavDriftStaysInBand() (gas: 340177)
[PASS] test_Fork_Mint_NavAtBandBoundary() (gas: 473676)
[PASS] test_Fork_Mint_QuotedNavDoesNotAffectMintedAmount() (gas: 476412)
[PASS] test_Fork_Mint_RevertDoubleMint() (gas: 336428)
[PASS] test_Fork_Mint_RevertExpireTimeTooFarOut() (gas: 84925)
[PASS] test_Fork_Mint_RevertNavAboveBand() (gas: 265983)
[PASS] test_Fork_Mint_RevertNavBelowBand() (gas: 266021)
[PASS] test_Fork_Mint_RevertWhenDepositorLacksMinterRole() (gas: 3221805)
[PASS] test_Fork_Mint_RevertWhenGatewayLacksMinterRole() (gas: 2657969)
[PASS] test_Fork_Mint_RevertWhenPaused() (gas: 98614)
[PASS] test_Fork_Mint_RevertWhenSharesRoundToZero() (gas: 299359)
[PASS] test_Fork_Mint_SharesAreFewerThanValue() (gas: 331840)
[PASS] test_Fork_Mint_StaleQuoteDoesNotDilute() (gas: 341255)
[PASS] test_Fork_Mint_ValueIsConserved() (gas: 351501)
[PASS] test_Fork_ProductionPreconditions() (gas: 115748)
[PASS] test_Fork_RateLimit_ExhaustionIsRetryable() (gas: 1094139)
Suite result: ok. 20 passed; 0 failed; 0 skipped; finished in 2.09s (5.85s CPU time)

Ran 22 test suites in 2.14s (5.16s CPU time): 438 tests passed, 0 failed, 0 skipped (438 total tests)
```

Code Coverage

File	% Lines	% Statements	% Branches	% Funcs
src/BTCPlusDepositor.sol	100.00% (52/52)	100.00% (59/59)	100.00% (12/12)	100.00% (8/8)
src/BTCPlusMainnetChannel.sol	97.92% (94/96)	98.10% (103/105)	92.31% (24/26)	100.00% (14/14)
src/BTCTxRegistry.sol	100.00% (24/24)	100.00% (20/20)	100.00% (6/6)	100.00% (4/4)
src/PauseController.sol	100.00% (20/20)	100.00% (14/14)	100.00% (4/4)	100.00% (4/4)
src/SolvBTCMainnetChannel.sol	98.41% (62/63)	98.53% (67/68)	94.74% (18/19)	100.00% (9/9)

File	% Lines	% Statements	% Branches	% Funcs
src/SolvBTCMintGateway.sol	100.00% (22/22)	100.00% (18/18)	100.00% (5/5)	100.00% (5/5)
src/utils/ChannelRateLimiter.sol	100.00% (67/67)	100.00% (62/62)	100.00% (13/13)	100.00% (11/11)
Total	99.13% (341/344)	99.13% (343/346)	96.47% (82/85)	100.00% (55/55)

Changelog

- 2026-09-11 - Initial Report
- 2026-09-23 - First Fix Review
- 2026-09-30 - Final Report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over $500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree

that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.



© 2026 – Quantstamp, Inc.